■ ESP32 Smart Grid Monitoring & Fault Analysis System

Comprehensive Technical Blueprint & Architectural Deep-Dive (Made by Yash)

Live App: smartgrid-monitoring.streamlit.app | **GitHub**: github.com/yashdeep043/smart-grid-monitoring

---

LAYER 1 — THE BIG PICTURE (Basic)

1.1 What is a Smart Grid, and why does it need continuous monitoring?

An **Electrical Grid** is a vast interconnected network designed to deliver electricity from power plants to consumers. A traditional grid operates **blindly**—power flows in one direction, and utility companies only learn about blackouts or line failures when angry customers call to complain.

A **Smart Grid** adds a layer of digital sensors, microcontrollers, wireless communication networks, and computer software on top of the physical wires and transformers.

Continuous monitoring is essential because electricity cannot be easily stored in large quantities within grid lines. Power generation and consumer demand must balance perfectly every millisecond. Unmonitored fluctuations cause:

- **Voltage Sags & Surges**: Damaging sensitive home appliances and industrial motors.
- **Thermal Line Overloading**: Overheating power lines and causing transformer explosions.
- **Unplanned Blackouts**: Taking hours or days for linesmen to locate broken lines physically.

1.2 What problem does this project solve compared to traditional grid monitoring?

1. **Manual Outage Hunting ➔ Instant 1-Second Fault Localization**: Traditional utilities rely on manual inspection to locate broken wires. This project uses OpenDSS impedance calculations to pinpoint exact fault locations (km) and calculate relay trip times (ms) instantly.
2. **Periodic Sampling ➔ Real-Time 1-Second Telemetry**: Legacy utility meters update every 15 minutes to 1 hour. This system captures 3-phase telemetry every **1 second** using lightweight MQTT messaging over Wi-Fi.
3. **Reactive Fixes ➔ Predictive AI Intelligence**: Traditional grids react *after* equipment fails. This project uses `IsolationForest` to flag micro-anomalies immediately and `RandomForestRegressor` to forecast energy demand 24 hours in advance.
4. **Prohibitive SCADA Costs ➔ Open-Source Accessibility**: Commercial SCADA software (Siemens Spectrum, ABB Ability) costs upwards of $500,000 per substation. This project provides enterprise-grade monitoring at **1% of the cost** using open-source Python, Pandapower, and $15 hardware nodes.

1.3 One-Paragraph End-to-End Summary

This project is an end-to-end cyber-physical Smart Grid monitoring and predictive analytics platform. Low-cost **ESP32 microcontrollers** equipped with **PZEM-004T AC sensors** sample 3-phase voltages, currents, active power, reactive power, power factor, and frequency every **1 second**, transmitting JSON telemetry over **MQTT** (broker.hivemq.com). A Python backend ingests the data into a **SQLite time-series database**, solves **Pandapower AC Newton-Raphson power flows** to monitor line loading and heat losses, executes **OpenDSS short-circuit impedance models** to evaluate relay trip times, and applies **Scikit-Learn Machine Learning** (`IsolationForest` for anomaly scoring and `RandomForestRegressor` for 24-hour demand forecasting). All metrics and interactive controls are rendered live on a cyber glassmorphism **Streamlit web dashboard** deployed at smartgrid-monitoring.streamlit.app.

LAYER 2 — HARDWARE & DATA ACQUISITION (Basic-Intermediate)

2.1 The ESP32 Microcontroller: Selection Rationale

The **ESP32** (designed by Espressif Systems) is a low-cost, low-power system-on-a-chip (SoC) microcontroller selected for key architectural reasons:

- **Dual-Core 32-bit Xtensa LX6 Microprocessor**: Runs at 240 MHz, allowing Core 0 to handle Wi-Fi/MQTT networking while Core 1 processes high-speed sensor sampling simultaneously without blocking.
- **Built-in 2.4 GHz Wi-Fi & Bluetooth**: Eliminates the need for external network shields.
- **Hardware UART Interfaces**: Features 3 hardware Serial UART ports, allowing direct digital communication with PZEM-004T power sensors via GPIO 16 (RX2) and GPIO 17 (TX2).
- **Ultra-Low Cost**: Costs under $5 per node compared to expensive industrial Remote Terminal Units (RTUs).

2.2 The PZEM-004T AC Sensor & Modbus-RTU Protocol

The **PZEM-004T (V3.0)** is an industrial-grade AC power measurement module measuring Voltage (80V - 260V), Current (0A - 100A via Split-Core CT ring), Active Power (0 - 23 kW), Frequency (45Hz - 65Hz), and Power Factor (0.00 - 1.00). It communicates with the ESP32 via UART TTL at 9600 baud using Modbus-RTU protocol.

2.3 Raw Sensor Payload (JSON Schema)

```
{
  "timestamp": "2026-07-26 00:30:00",
  "node_id": "ESP32_SUBSTATION_01",
  "voltage_a": 230.85,
  "voltage_b": 229.40,
  "voltage_c": 230.15,
  "current_a": 14.25,
  "current_b": 13.80,
  "current_c": 14.05,
  "active_power": 9.25,
  "reactive_power": 1.82,
  "power_factor": 0.95,
  "frequency": 50.02,
  "status": "NORMAL"
}
```

LAYER 3 — COMMUNICATION LAYER (Intermediate)

3.1 MQTT Architecture & Publish-Subscribe Model

MQTT (Message Queuing Telemetry Transport) is an ISO-standard lightweight messaging protocol. Publishers (ESP32) send JSON payloads to topics (esp32/smartgrid/telemetry) at broker.hivemq.com:1883, and Subscribers (database.py / app.py) consume the data stream in real time.

3.2 Why MQTT over HTTP REST?

- **Bandwidth Efficiency**: MQTT packet headers are tiny (2 bytes) compared to HTTP headers (~500 bytes).
- **Decoupled Architecture**: Publishers and subscribers operate independently without knowing each other's IP addresses.
- **Persistent Socket Connection**: Keeps an open TCP connection, eliminating repeated HTTP handshakes.

LAYER 4 — DATA STORAGE (Intermediate)

4.1 SQLite Selection Rationale

SQLite is a self-contained, serverless, zero-configuration relational database stored in smart_grid.db. It eliminates external database server overhead while handling thousands of reads and writes cleanly.

4.2 Database Schemas (database.py)

- **telemetry**: Stores 1-second sensor readings (V, I, P, Q, PF, f, status).
- **fault_events**: Stores short-circuit simulation logs (fault current kA, dip %, trip ms, severity).
- **anomalies**: Stores IsolationForest ML outlier flags and descriptions.

LAYER 5 — POWER SYSTEM SIMULATION (Intermediate-Advanced)

5.1 Pandapower & Role in System Architecture

Pandapower is an open-source power system solver. In grid_simulation.py, it enforces Kirchhoff's Laws across a 7-bus 11kV/0.4kV distribution network.

5.2 Mathematical Formulation of Newton-Raphson Power Flow

Solves non-linear AC power balance equations iteratively using the Jacobian Matrix (J):

P_i = V_i *SUM [V_j* (G_ij *cos(theta_ij)* + *B_ij* sin(theta_ij))]

Q_i = V_i *SUM [V_j* (G_ij *sin(theta_ij)* - *B_ij* cos(theta_ij))]

5.3 Operational Grid Thresholds

- **Bus Voltage Magnitude**: Normal (0.95 - 1.05 p.u.). Violation if < 0.95 p.u. (Sag) or > 1.05 p.u. (Swell).
- **Line Loading Capacity**: Normal (< 75%), Overload (> 90%).

LAYER 6 — FAULT ANALYSIS ENGINE (Advanced)

6.1 Fault Classification & Impedance Math

Simulates 3PH Symmetrical, Single Line-to-Ground (SLG), and Line-to-Line (LL) faults. Short-circuit currents are calculated using complex sequence impedances (Z1, Z0).

6.2 IEC Extremely Inverse Relay Trip-Time Formula

Follows the IEC 60255 inverse-time trip characteristic:

t_trip = 80 / [(I_fault / I_pickup)^2 - 1] seconds

Larger fault currents trigger faster trip times (e.g. 5kA trips in 40 ms; 600A trips in 4.2 s).

LAYER 7 — AI / MACHINE LEARNING LAYER (Advanced)

7.1 Isolation Forest Anomaly Detection Engine

Uses an ensemble of 100 decision trees to isolate telemetry outliers. Sags (< 180V) and surges (> 100A) isolate near tree roots, receiving high negative anomaly scores.

7.2 Random Forest Load Forecasting Engine

Uses RandomForestRegressor(n_estimators=50) mapping (hour, day of week) to active power demand (kW) with a 90% confidence interval band.

LAYER 8 — DASHBOARD & SYSTEM INTEGRATION (Advanced)

8.1 Streamlit Dashboard Architecture

Renders 5 interactive tabs (Telemetry, Power Flow, Fault Injection, AI Forecast, Architecture) running live at smartgrid-monitoring.streamlit.app using WebSockets state management.

LAYER 9 — ARCHITECTURE & DESIGN DECISIONS (Expert / Interview-Level)

9.1 Prototype vs. Industrial Production Roadmap

To scale onto real distribution poles: replace Wi-Fi with 4G LTE / LoRaWAN modules, switch public HiveMQ broker to private Mosquitto with TLS/SSL encryption, and upgrade SQLite to InfluxDB time-series database.

■ ELEVATOR PITCH SUMMARY

"I designed and built the ESP32 Smart Grid Monitoring & Fault Analysis System, an end-to-end cyber-physical platform that digitizes power distribution networks at 1% of the cost of commercial SCADA systems. The system captures real-time 3-phase AC telemetry every 1 second via ESP32 microcontrollers and MQTT. Its hybrid Python engine integrates Pandapower AC Newton-Raphson power flow calculations, OpenDSS short-circuit fault impedance models with IEC overcurrent relay trip timing, and Scikit-Learn AI (IsolationForest for automated anomaly detection and RandomForestRegressor for 24-hour demand forecasting). The entire platform is deployed live on Streamlit Cloud at smartgrid-monitoring.streamlit.app."